Nicla Sense ME User Manual

Learn about the hardware and software features of the Arduino® Nicla Sense ME.

Author · Christopher Mendez Last revision · 07/17/2024

Overview

This user manual will guide you through a practical journey covering the most interesting features of the Arduino Nicla Sense ME. With this user manual, you will learn how to set up, configure and use this Arduino board.

Hardware and Software Requirements

Hardware Requirements

- Nicla Sense ME (x1)
- Micro USB cable (x1)

Software Requirements

- Arduino IDE 1.8.10+, Arduino IDE 2.0+, or Arduino Cloud Editor

To create custom Machine Learning models, the Machine Learning Tools add-on

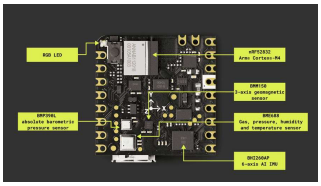
not have an Arduino Cloud account, you will need to create one first.

Product Overview

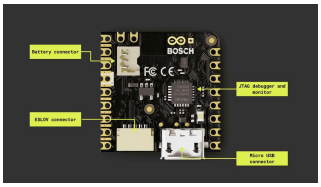
The Arduino® Nicla Sense ME is one of the most compact designs of the Arduino portfolio, housing an array of high-quality industrial-grade sensors in a remarkably small package. This versatile device allows you to accurately monitor essential process parameters like temperature, humidity, and motion, and share data through Bluetooth® Low Energy connectivity, enabling endless low-power smart sensing applications.

Board Architecture Overview

The Nicla Sense ME features a robust and efficient architecture that integrates a range of sensors packed into a tiny footprint. It features four industrial grade Bosch sensors that can accurately measure rotation, acceleration, pressure, humidity, temperature, air quality and CO2 levels.



Nicla Sense ME main components (top view)



Here is an overview of main components of the board, as shown in the images above:

- **Microcontroller**: at the heart of the Nicla Sense ME is the nRF52832, a powerful and versatile System-on-Chip (SoC) from Nordic® Semiconductor. The nRF52832 is built around a 32-bit Arm® Cortex®-M4 processor running at 64 MHz.
- **Onboard advanced motion sensors**: the board features the BHI260AP, a smart IMU that includes a 3-axis accelerometer and a 3-axis gyroscope. It is trained with Machine Learning algorithms able to perform step counting, position tracking, and activity recognition. The board also features the BMM150, a compact geomagnetic sensor from Bosch® Sensortec including a 3-axis magnetometer.
- **Onboard environment sensors**: the Nicla Sense ME is equipped with the BME688, this is the first gas sensor with Artificial Intelligence (AI) and integrated high-linearity and high-accuracy pressure, humidity and temperature sensors. The gas sensor can detect Volatile Organic Compounds (VOCs), volatile sulfur compounds (VSCs) and other gases, such as carbon monoxide and hydrogen, in the part per billion (ppb) range.
- **Wireless connectivity**: the board supports Bluetooth® Low Energy connectivity, provided by the ANNA-B112 module developed by u-blox®. This compact, high-performance Bluetooth® Low Energy module allows the Nicla Sense ME to communicate wirelessly with other devices and systems.
- **Power management**: the Nicla

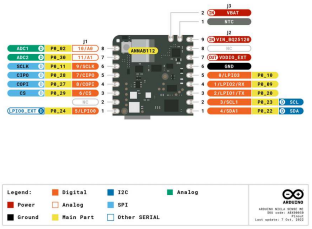
efficient power management features that ensure minimal energy consumption even when using always-on motion recognition and environment analysis sensors. The Nicla Sense ME features the BQ25120 from Texas Instruments®, a highly integrated battery charge management integrated circuit (IC) designed for wearables and Internet of Things (IoT) devices.

Board Core and Libraries

The **Arduino Mbed OS Nicla Boards** core contains the libraries and examples you need to work with the board's components, such as its IMU, magnetometer, and environment sensor. To install the Nicla boards core, navigate to **Tools > Board > Boards Manager** or click the Boards Manager icon in the left tab of the IDE. In the Boards Manager tab, search for `Nicla Sense ME` and install the latest `Arduino Mbed OS Nicla Boards` version.

Installing the Arduino Mbed OS Nicla Boards core in the Arduino IDE bootloader

Pinout



Nicla Sense ME pinout

The full pinout is available and downloadable as PDF from the link below:

Nicla Sense ME pinout

Datasheet

The complete datasheet is available and downloadable as PDF from the link below:

Nicla Sense ME datasheet

Schematics

The complete schematics are available and downloadable as PDF from the link below:

Nicla Sense ME schematics

STEP Files

The complete STEP files are available and downloadable from the link below:

Nicla Sense ME STEP files

First Use

Powering the Board

The Nicla Sense ME can be powered by:

- A Micro USB cable (not included).
- An external **5V power supply** connected to `VIN_BQ25120` pin (please, refer to the board pinout section of the user manual).
- A **3.7V Lithium Polymer (Li-Po) battery** connected to the board through the onboard battery connector; the manufacturer part number of the battery connector is BM03B-ACHSS and its matching receptacle manufacturer part number is ACHR-03V-S. The **recommended minimum battery capacity for the Nicla Sense ME is 200 mAh**. A Li-Po battery with an integrated NTC thermistor is also recommended for thermal protection.
- The onboard **ESLOV connector**, which has a dedicated 5V power line.

Nicla Sense ME battery powered

Hello World Example

Let's program the Nicla Sense ME with the classic `hello world` example typical of the Arduino ecosystem: the `Blink` sketch. We will use this example to verify that the board is correctly connected to the Arduino IDE and that the Nicla Sense ME core and the board itself

Copy and paste the code below into a new sketch in the Arduino IDE.

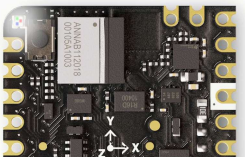
```
#include "Nicla_System.h"

void setup() {
  // Initialize LED_BUILT
  pinMode(LED_BUILTIN, OU
}

void loop() {
  // Turn the built-in LE
  digitalWrite(LED_BUILTI
  delay(1000);
  // Turn the built-in LE
  digitalWrite(LED_BUILTI
  delay(1000);
}
```

For the Nicla Sense ME, the `LED_BUILTIN` macro represents **all the LEDs** of the built-in RGB LED of the board, shining in **white**.

To upload the code to the Nicla Sense ME, click the **Verify** button to compile the sketch and check for errors; then click the **Upload** button to program the board with the sketch.

Uploading a sketch to the Nicla Sense ME in the Arduino IDE

You should now see all the LEDs of the built-in RGB LED turn on for one second, then off for one second, repeatedly.



Battery Charging

One of the characteristic features of the Nicla Sense ME is power management, the BQ25120 battery charger IC is configurable by the user, which means that its charging parameters can be customized by software. We listed the main ones below:

Enable charging: If you are powering the board with a rechargeable battery, you may want it to be recharged, the IC lets you enable the charging function by calling `nicla::enableCharging()`.

Battery charging current: A safe default charging current value that works for most common LiPo batteries is 0.5C, which means charging at a rate equal to half of the battery's capacity. For example, a 200mAh battery could be charged at 100mA (0.1A).

The desired current must be set as the parameter of the enabling function: `nicla::enableCharging(100)`.

When the function parameter is left blank, the default current is 20mA.

Battery NTC: If your battery

by calling this function:

```
nicla::setBatteryNTCEnabled(true)
```

, if not, set the argument to *false*.

Battery maximum charging time: To get an estimation of the charging time, you can use the following formula:

```
Charging time (in hours) = (Battery capacity in mAh) / (0.8 * Charging current in mA)
```

This formula takes into account that the charging process is approximately 80% efficient (hence the 0.8 factor). This is just a rough estimate, and actual charging time may vary depending on factors like the charger, battery quality, and charging conditions.

To set a charging time of nine hours, define it as follows:

```
nicla::configureChargingSafetyTimer (ChargingSafetyTimerOption::NineHours)
```

Get the battery voltage: To monitor the battery voltage you just need to store the returned value of the following function in a *float* variable:

```
float currentVoltage = nicla::getCurrentBatteryVoltage();
```

Get the power IC operating status: In order to know if the battery is fully charged, charging, or presents any error, you can check its status using this code block:

```
auto operatingStatus = ni

    switch(operatingStatu
      case OperatingStatu
      nicla::leds.setColo
        break;
      case OperatingStatu
        nicla::leds.setCo

        // This will stop
        nicla::disableCha
        break;
      case OperatingStatu
        nicla::leds.setCo
        break;
      case OperatingStatu
        nicla::leds.setCo
        break;
      default:
        nicla::leds.setCo
        break;
    }
```

To extend your knowledge on this topic, refer to the board examples by navigating to "**File > Examples > Nicla_Sense_System**", and choose between both examples:

`NiclaSenseME_BatteryStatus`

`NiclaSenseME_BatteryChargingSimple`

Pins

Analog Pins

The Nicla Sense ME has **two analog input pins**, mapped as follows:

Microcontroller Pin	Arduino Pin Mapping
ADC1/P0_02	A0
ADC2/P0_30	A1

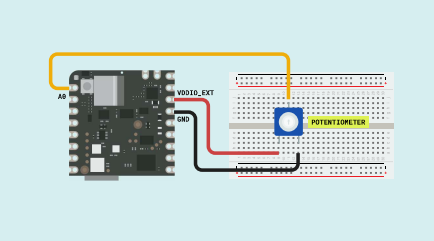
programming language.

Nicla boards ADC can be configured to 8, 10 or 12 bits defining the argument of the following function respectively (default is 10 bits):

```
analogReadResolution(12);
```

> The Nicla boards ADC reference voltage is fixed to 1.8V, this means that it will map the ADC range from 0 to 1.8 volts.

The example code shown below reads the analog input value from a potentiometer connected to `A0` and displays it on the IDE Serial Monitor. To understand how to properly connect a potentiometer to the Nicla Sense ME pins, take the following image as a reference:



```
#include "Nicla_System.h"

int sensorPin = A0;    //
int sensorValue = 0;   //

void setup() {

  analogReadResolution(12
  nicla::begin();
  Serial.begin(115200);
}

void loop() {
  // read the value from
  sensorValue = analogRea
  // print the value
  Serial.println(sensorVa
  delay(1000);
}
```

The ADC inputs support 3.3V even when the ADC reference is 1.8V. In this perspective, the ADC will not sense any change from 1.8V and above.

Digital Pins

The Nicla Sense ME has **twelve digital pins**, mapped as follows:

Microcontroller Pin	Arduino Pin Mapping
P0_10	0
P0_09	1
P0_20	2
P0_23	3
P0_22	4
P0_24	5
P0_29	6
P0_27	7
P0_28	8

Microcontroller Pin	Arduino Pin Mapping
P0_30	A1

Notice that analog pins `A0` and `A1` (`P0_02` and `P0_30`) can also be used as digital pins. Please, refer to the **board pinout section** of the user manual to check their location.

The digital pins of the Nicla Sense ME can be used as inputs or outputs through the built-in functions of the Arduino programming language.

> The Nicla Sense ME digital I/O's are low power, so to drive output devices like LEDs, resistive loads, buzzers, etc, it is recommended to use a MOSFET driver or a buffer to guarantee the required current flow. Learn more about the Nicla I/O's considerations **here**.

The configuration of a digital pin is done in the `setup()` function with the built-in function `pinMode()` as shown below:

```
// Pin configured as an in
pinMode(pin, INPUT);

// Pin configured as an ou
pinMode(pin, OUTPUT);

// Pin configured as an in
pinMode(pin, INPUT_PULLUP)
```

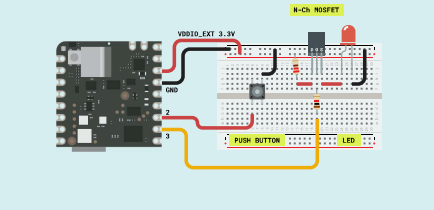
The state of a digital pin, configured as an input, can be read using the built-in function `digitalRead()` as shown below:

```
// Read pin state, store v
state = digitalRead(pin);
```

The state of a digital pin, configured as an output, can be changed using the built-in function `digitalWrite()` as shown below:

```
// Set pin on
digitalWrite(pin, HIGH);

// Set pin off
digitalWrite(pin, LOW);
```

The example code shown below uses digital pin `3` to control an LED and reads the state of a button connected to digital pin `2`:



```
// Define button and LED
int buttonPin = 2;
int ledPin = 3;

// Variable to store the
int buttonState = 0;

void setup() {
  // Configure button ar
  pinMode(buttonPin, INP
  pinMode(ledPin, OUTPUT

  // Initialize Serial c
  Serial.begin(115200);
}

void loop() {
  // Read the state of t
  buttonState = digitalR

  // If the button is pr
  if (buttonState == LOW
    digitalWrite(ledPin,
    Serial.println("- Bu
  } else {
    // If the button is
    digitalWrite(ledPin,
    Serial.println("- Bu
```

PWM Pins

Most digital and analog pins of the Nicla Sense ME can be used as PWM (Pulse Width Modulation) pins. This functionality can be used with the built-in function `analogWrite()` as shown below:

```
analogWrite(pin, value);
```

By default, the output resolution is 8 bits, so the output value should be between 0 and 255. To set a greater resolution, do it using the built-in function `analogWriteResolution` as

```
analogWriteResolution(bits
```

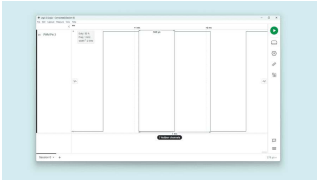
Using this function has some limitations, for example, the PWM signal frequency is fixed at 500 Hz, and this could not be ideal for every application.

PWM output signal using analogWrite()

If you need to work with a higher frequency PWM signal, you must do it by working with the PWM peripheral at a lower level as shown in the example code below:

```
#include "nrfx_pwm.h"

static nrfx_pwm_t pwm1 =

static uint16_t /*const*

static nrf_pwm_sequence_
  .values = { .p_common
  .length = NRF_PWM_VALU
  .repeats = 0,
  .end_delay = 0
};

void setup() {

  nrfx_pwm_config_t conf
    .output_pins = {
      32 + 23,  // Nicla
    },
    .irq_priority = APP_
    .base_clock = NRF_PW
    .count_mode = NRF_PW
    .top_value = 1000,
    .load_mode = NRF_PWM
    .step_mode = NRF_PWM
  };

  nrfx_pwm_init(&pwm1, &
```

The code above results in a 1KHz square waveform with a 50% duty cycle as in the image below. The frequency is defined by the `.base_clock` and `.top_value` variables, and the duty cycle by the `seq1_values` variable.

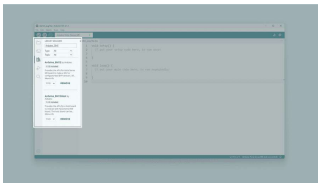


PWM output signal using the PWM at a lower level

Onboard Sensors

allow you to capture and process environmental and motion data via a 6-axis IMU, a 3-axis magnetometer and a gas, temperature, humidity and pressure sensor. The onboard sensors can be used for developing various applications, such as activity recognition, and environmental monitoring.

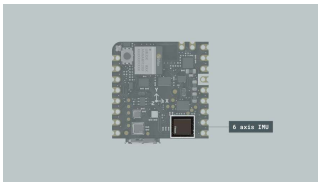
To read from any of these sensors you need to install the Arduino_BHY2 and Arduino_BHY2Host libraries. These can be found in the Arduino IDE library manager. To do so in the IDE, select it from the left side menu, now search for Arduino_BHY and click on the install button.



BHY2 library install

IMU

The Nicla Sense ME features an advanced IMU, which allows the board to sense motion. The IMU on the board is the BHI260AP from Bosch®. It consists of a 3-axis accelerometer and a 3-axis gyroscope. They can provide information about the board's motion, orientation, and rotation in a 3D space. The BHI260AP, apart from being able to do raw movement measurements, is equipped with pre-trained machine-learning models that recognize activities right out of the box.



Nicla Sense ME onboard IMU

The example code below shows how to get acceleration and angular velocity data from the onboard IMU and stream it to the Arduino IDE Serial Monitor and Serial Plotter.

```
#include "Arduino.h"
#include "Arduino_BHY2.h"

SensorXYZ accel(SENSOR_ID_ACC);
SensorXYZ gyro(SENSOR_ID_GYRO);


void setup() {
  Serial.begin(115200);
  while (!Serial)
    ;

  BHY2.begin();

  accel.begin();
  gyro.begin();
}

void loop() {
  static auto printTime = millis();

  // Update function should be called frequently
  BHY2.update();

  if (millis() - printTime >= 20) {
    printTime = millis();

    // Accelerometer values
```

Accelerometer and gyroscope output in the serial plotter

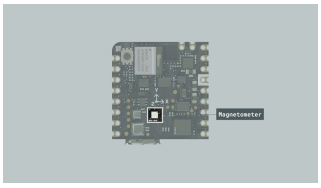
To take advantage of the IMU pre-trained ML capabilities, we can use the *Activity Recognition* class. The following example code enables your Nicla Sense ME to classify movements from different daily activities:

- Standing still
- Walking
- Running
- On bicycle
- In vehicle
- Tilting
- In vehicle still

```
#include "Nicla_System.h
#include "Arduino_BHY2.h

SensorActivity active(SE

unsigned long previousMi

const long interval = 10

void setup() {

  Serial.begin(115200);
  nicla::begin();
  BHY2.begin(NICLA_I2C);
  active.begin();
}

void loop() {

  BHY2.update();

  unsigned long currentM

  if (currentMillis - pr

    previousMillis = cur
    Serial.println(Strin

```

Magnetometer

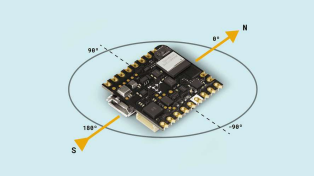
The Nicla Sense ME is equipped with an onboard magnetometer, which allows the board to sense orientation and magnetic fields. The BMM150 enables measurements of the magnetic field in three perpendicular axes. Based on Bosch's proprietary FlipCore technology, the performance and features of BMM150 are carefully tuned to perfectly match the demanding requirements of 3-axis mobile applications such as electronic compass, navigation, or augmented reality.



Nicla Sense ME onboard magnetometer

In the example code below, the magnetometer is used as a compass measuring the heading orientation in degrees.

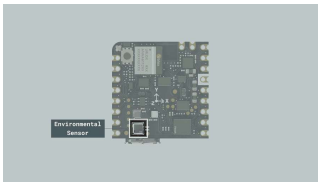
```
#include "Nicla_System.h
#include "Arduino_BHY2.h
#include "Math.h"

SensorXYZ magnetometer(S

float heading = 0;

unsigned long previousMi

const long interval = 10

void setup() {

  Serial.begin(115200);
  nicla::begin();
  BHY2.begin(NICLA_I2C);
  magnetometer.begin();

}

void loop() {
  BHY2.update();

  unsigned long currentM

  if (currentMillis - pr
```



Compass orientation

Environmental Sensor

The Arduino Nicla Sense ME can perform environmental monitoring via the Bosch BME688 sensor. This enables pressure, humidity, temperature as well as gas detection. The gas sensor can detect Volatile Organic Compounds (VOCs), volatile sulfur compounds (VSCs), and other gases, such as carbon monoxide and hydrogen, in the part per billion (ppb)



Nicla Sense ME onboard environmental sensor

The BME688 lets you measure pressure, humidity, temperature, and gas sensor resistance, thanks to a proprietary solution from Bosch called BSEC. In addition, the system is capable of providing numerous useful outputs such as:

- Index for Air Quality (IAQ)
- CO_2 equivalents
- b-VOC equivalents
- Gas %

To extract these measurements from the sensor use the below example code:

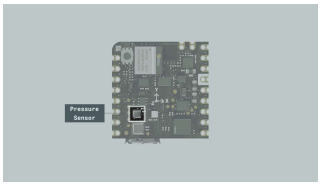
```
#include "Nicla_System.h
#include "Arduino_BHY2.h


unsigned long previousMi

const long interval = 1

SensorBSEC bsec(SENSOR_I

void setup() {

  Serial.begin(115200);
  nicla::begin();
  BHY2.begin(NICLA_I2C);
  bsec.begin();
}

void loop() {

  BHY2.update();

  unsigned long currentM

  if (currentMillis - pr

    previousMillis = cur
```

When using the BSEC sensor class, be aware that the system will need several minutes to start providing IAQ and CO_2 measurements due to some mandatory internal calibrations.

Pressure Sensor

The Nicla Sense ME can accurately measure pressure thanks to its digital pressure sensor (BMP390). Its operating range is from 300 to 1250 hPa, which makes it perfect for a variety of applications, including:

- Weather forecast
- Outdoor navigation
- Vertical velocity indication



Nicla Sense ME onboard barometric pressure sensor

To use this sensor in standalone mode, you can leverage the example code below:

```
#include "Nicla_System.h
#include "Arduino_BHY2.h


unsigned long previousMi

const long interval = 10

Sensor pressure(SENSOR_I

void setup() {

  Serial.begin(115200);
  nicla::begin();
  BHY2.begin(NICLA_I2C);
  pressure.begin();
}

void loop() {

  BHY2.update();

  unsigned long currentM

  if (currentMillis - pr

    previousMillis = cur
```

To learn how to work with every sensor class and predefined objects to get all the needed readings, go to our Nicla Sense ME Cheat Sheet.

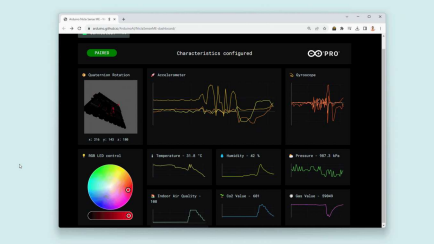
On-Board Sensors WebBLE Dashboard

A very interesting way to test the Nicla Sense ME onboard sensors all at once is through the WebBLE dashboard demo.

Enable your PC Bluetooth connection and go to the **dashboard link**, add this **firmware** to your sketchbook in the Arduino Cloud or download it to use it locally.

Upload the code to your Nicla Sense ME and now you are ready to start monitoring the variables through the WebBLE dashboard.

Click on "CONNECT", search for your board and pair it.

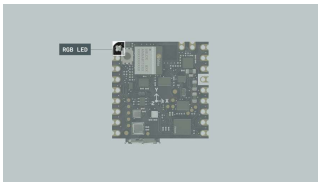


Follow this **dedicated guide** to get more details.

Actuators

RGB LED

The Nicla Sense ME features a built-in I2C RGB LED that can be a visual feedback indicator for the user. The LED is connected through the boards' I2C port; therefore, specific functions must be used to operate the LED colors.



Built-in RGB LED

To use the RGB LED, include the `Nicla System` header:

```
// Include the Nicla Syste
#include "Nicla_System.h"
```

Since the functions are scoped under a class name called `nicla`, you must explicitly write it before each statement. To initialize the board's built-in RGB LED along with the Nicla system inside the void `setup()` function:

```
void setup() {
  // Initialize the Nicla
  nicla::begin();
  nicla::leds.begin();
}
```

The LED can be set to the desired RGB value using red, green and blue components or by using one of the following predefined colors:

- off
- red
- green
- blue
- yellow
- magenta
- cyan

To set the LED to a predefined color (e.g. green or blue):

```
// Set the LED color to gr
nicla::leds.setColor(green
delay(1000);

// Set the LED color to bl
nicla::leds.setColor(blue)
delay(1000);
```

To turn off the built-in RGB LED:

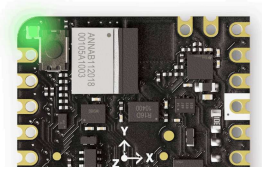
```
// Turn off the LED
nicla::leds.setColor(off);
```

You can also choose a value between 0 and 255 for each color component (red, green, or blue) to set a custom color:

```
// Define custom color va
int red = 234;
int green = 72;
int blue = 122;

// Set the LED to the cus
nicla::leds.setColor(red,
delay(1000);

// Turn off the LED and w
nicla::leds.setColor(off)
delay(1000);
```

Here you can find a complete example code to blink the built-in I2C RGB LED of the Nicla Sense ME:

```
// Include the Nicla Syst
#include "Nicla_System.h"

void setup() {
  // Initialize the Nicla
  nicla::begin();
  nicla::leds.begin();
}

void loop() {
  // Set the LED color to
  nicla::leds.setColor(gr
  delay(1000);

  // Turn off the LED and
  nicla::leds.setColor(of
  delay(1000);
}
```



Communication

This section of the user manual covers the different communication protocols that are supported by the Nicla Sense ME board, including the Serial Peripheral Interface (SPI), Inter-Integrated Circuit (I2C), Universal Asynchronous Receiver-Transmitter (UART), and Bluetooth® Low Energy; communication via the onboard ESLOV connector is also explained in this section. The Nicla Sense ME features dedicated pins for each communication protocol, simplifying the connection and communication with different components, peripherals, and sensors.

The Nicla Sense ME supports SPI communication, which allows data transmission between the board and other SPI-compatible devices. The pins used in the Nicla Sense ME for the SPI communication protocol are the following:

Microcontroller Pin	Arduino Pin Mapping
CS/P0_29	SS or 6
COPI/P0_27	MOSI or 7
CIPO/P0_28	MISO or 9
SCLK/P0_11	SCK or 8

Please, refer to the board pinout section of the user manual to localize them on the board.

Include the `SPI` library at the top of your sketch to use the SPI communication protocol. The SPI library provides functions for SPI communication:

```
#include <SPI.h>
```

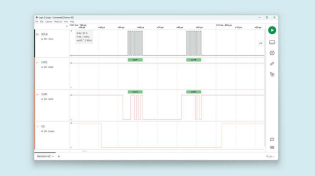
In the `setup()` function, initialize the SPI library, define and configure the chip select (`CS`) pin:

```
void setup() {
  // Set the chip select
  pinMode(SS, OUTPUT);

  // Pull the CS pin HIGH
  digitalWrite(SS, HIGH);

  // Initialize the SPI c
  SPI.begin();
}
```

To transmit data to an SPI-compatible device, you can use the following commands:

```
// Replace with the targe
byte address = 0x35;

// Replace with the value
byte value = 0xFA;

// Pull the CS pin LOW to
digitalWrite(SS, LOW);

// Send the address
SPI.transfer(address);

// Send the value
SPI.transfer(value);

// Pull the CS pin HIGH t
digitalWrite(SS, HIGH);
```

The example code above should output this:



SPI logic data output

I2C

The Nicla Sense ME supports I2C communication, which allows data transmission between the board and other I2C-compatible devices. The pins used in the Nicla Sense ME for the I2C communication protocol are the following:

Microcontroller Pin	Arduino Pin Mapping

Microcontroller Pin	Arduino Pin Mapping
P0_22	SDA or 4

Please, refer to the board pinout section of the user manual to localize them on the board. The I2C pins are also available through the Nicla Sense ME ESLOV connector.

To use I2C communication, include the `Wire` library at the top of your sketch. The `Wire` library provides functions for I2C communication:

```
#include <Wire.h>
```

In the `setup()` function, initialize the I2C library:

```
// Initialize the I2C comm
Wire.begin();
```

To transmit data to an I2C-compatible device, you can use the following commands:

```
// Replace with the targe
byte deviceAddress = 0x35

// Replace with the appro
byte instruction = 0x00;

// Replace with the value
byte value = 0xFA;

// Begin transmission to
Wire.beginTransmission(de

// Send the instruction b
Wire.write(instruction);

// Send the value
Wire.write(value);

// End transmission
Wire.endTransmission();
```

The output data should look like the image below, where we can see the device address data frame:

I2C output data

To read data from an I2C-compatible device, you can use the `requestFrom()` function to request data from the device and the `read()` function to read the received bytes:

```
// The target device's I2
byte deviceAddress = 0x1;

// The number of bytes to
int numBytes = 2;

// Request data from the
Wire.requestFrom(deviceAd

// Read while there is da
while (Wire.available())
  byte data = Wire.read()
}
```

UART

The pins used in the Nicla Sense ME for the UART communication protocol are the following:

Microcontroller Pin	Arduino Pin Mapping
P0_09	RX or 1
P0_20	TX or 2

Please, refer to the board pinout section of the user manual to localize them on the board.

To begin with UART communication, you will need to configure it first. In the `setup()` function, set the baud rate (bits per second) for UART communication:

```
// Start UART communicatio
Serial1.begin(115200);
```

To read incoming data, you can use a `while()` loop to continuously check for available data and read individual characters. The code shown below

String variable and processes the data when a line-ending character is received:

```
// Variable for storing i
String incoming = "";

void loop() {
  // Check for available
  while (Serial1.availabl
    // Allow data bufferi
    delay(2);
    char c = Serial1.read

    // Check if the chara
    if (c == '\n') {
      // Process the rece
      processData(incomin

      // Clear the incomi
      incoming = "";
    } else {
      // Add the characte
      incoming += c;
    }
  }
}
```

To transmit data to another device via UART, you can use the `write()` function:

```
// Transmit the string "He
Serial1.write("Hello world
```

You can also use the `print` and `println()` to send a string without a newline character or followed by a newline character:

```
// Transmit the string "He
Serial1.print("Hello world
// Transmit the string "He
Serial1.println("Hello wor
```

Bluetooth® Low Energy

To enable Bluetooth® Low Energy communication on the Nicla Sense ME, you can use the **ArduinoBLE library**.

For this BLE application example, we are going to monitor the Nicla Sense ME battery level. Here is an example of how to use the ArduinoBLE library to achieve it:

```
#include "Nicla_System.
#include <ArduinoBLE.h>

// Bluetooth® Low Energ
BLEService batteryServi

// Bluetooth® Low Energ
BLEUnsignedCharCharacte


int oldBatteryLevel = 0
long previousMillis = 0

void blePeripheralDisco
  nicla::leds.setColor(
  Serial.println("Devic
}

void blePeripheralConne
  nicla::leds.setColor(
  Serial.println("Devic
}

void setup() {
  Serial.begin(9600);
  while (!Serial)
    ;

```

The example code shown above creates a Bluetooth® Low Energy service and characteristics according to the BLE standard for transmitting a battery percentage value read by Nicla Sense ME power management IC.

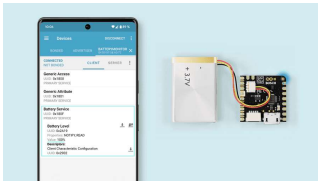
The code begins by importing all the necessary libraries and defining the Bluetooth® Low Energy service and characteristics for a battery-level application.

Description	ID
Battery Service	180F
Battery Level Characteristic	2A19

In the `setup()` function, the code initializes the Nicla Sense ME board and sets up the Bluetooth® Low Energy service and characteristics; then, it begins advertising the defined Bluetooth® Low Energy service.

A Bluetooth® Low Energy connection is constantly verified in the `loop()` function; when a central device connects to the Nicla Sense ME, its built-in LED is turned on blue. The code then enters into a loop that constantly reads the battery percent. It also prints it to the Serial Monitor and transmits it to the central device over the defined Bluetooth® Low Energy characteristic.

Using the nRF Connect app (available for Android and iOS) you can easily connect to your Nicla Sense ME and



Battery level monitored from the nRF Connect app

ESLOV Connector

The Nicla Sense ME board features an onboard ESLOV connector meant as an **extension** of the I2C communication bus. This connector simplifies the interaction between the Nicla Sense ME and various sensors, actuators, and other modules, without soldering or wiring.

Nicla Sense ME built-in ESLOV connector

The ESLOV connector is a small 5-pin connector with a 1.00 mm pitch; the mechanical details of the connector can be found in the connector's datasheet.

The manufacturer part number of the ESLOV connector is SM05B-SRSS and its matching receptacle manufacturer part number is SHR-05V-S-B.

The pin layout of the ESLOV connector is the following:

1. VCC_IN (5V input)
2. INT
3. SCL

Arduino Cloud

The Nicla Sense ME does not have built-in Wi-Fi®, so it can not be directly connected to the internet. For this, we need to use a Wi-Fi® capable Arduino board as a host for the Nicla.

In this example, a Portenta C33 will be used as a gateway to forward Nicla Sense ME sensors data to the Arduino Cloud.

Nicla Sense ME Setup

The **Nicla Sense ME** will be listening to the Host board to send back the required data. This is all automated via the libraries **Arduino_BHY2** and **Arduino_BHY2Host**.

The code is available inside the examples provided with the Arduino_BHY2 Library. Open it by going to **Examples > Arduino_BHY2 > App.ino**.

```
#include "Arduino.h"
#include "Arduino_BHY2.h"

// Set DEBUG to true in o
#define DEBUG false

void setup()
{
#if DEBUG
  Serial.begin(115200);
  BHY2.debug(Serial);
#endif

  BHY2.begin();   // this
}

void loop()
{
  // Update and then slee
  BHY2.update(100);
}
```

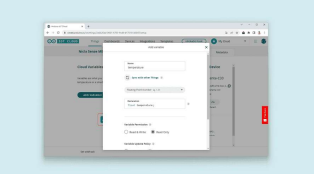
Upload the sketch above to the Nicla Sense ME using the Arduino IDE.

Arduino Cloud Setup

To start using the Arduino Cloud, we first need to log in or sign up.

Once in, it is time to configure your Portenta C33. For this, follow this section on the Portenta C33 user manual.

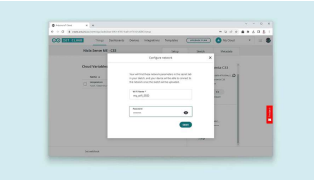
With a Thing already created, add a variable, in this case, "temperature" float type.



Adding the temperature variable to the Thing

Once the variable is added, let's

board, for this, click on your Thing Network setting and add your Wi-Fi® SSID and password:



Wi-Fi® credentials

It is time to open the automatically generated sketch and modify the code. It should be replaced by the following:

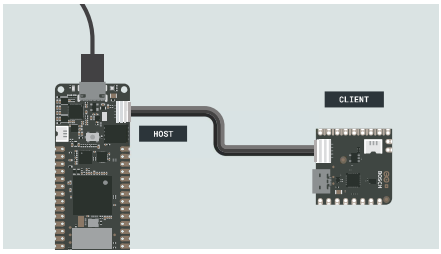
```
#include "thingPropertie
#include "Arduino_BHY2Ho

Sensor tempSensor(SENSOR

void setup() {
  Serial.begin(9600);
  delay(1500);

  Serial.print("Configur
  // Defined in thingPro
  initProperties();

  // Connect to Arduino
  ArduinoCloud.begin(Ard

  Serial.println("Connec
  while (ArduinoCloud.co
    ArduinoCloud.update(
    delay(500);
  }

  delay(1500);

  Serial.println("Initia
  BHY2Host.begin(false,

  tempSensor.configure(1
```

If you are interested in using a different sensor from the onboard options of the Nicla Sense ME, check all the sensors IDs on this **list**

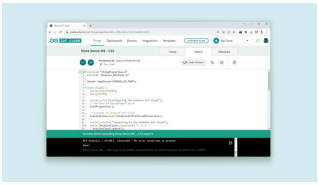
Portenta C33 Setup

Before uploading the code to the Portenta C33 code ready on the Arduino Cloud, let's connect everything together.

Using the ESLOV cable included with the Nicla Sense ME, connect both boards by their respective connectors as shown below:

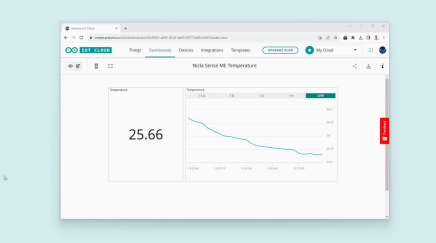


Upload the code to the Portenta C33 by connecting it to your computer using a USB cable and clicking on the upload button in the IoT Cloud editor.



Uploading the sketch to the Portenta C33

Finally, after searching for and connecting to your Wi-Fi® network, it will gather the temperature information from the Nicla Sense ME. Every 2 seconds it will forward it to the Cloud where we can monitor it



Bluetooth® Low Energy Connection

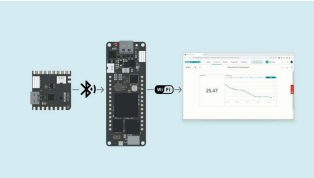
For Bluetooth® communication, substitute the line of code

```
BHY2Host.begin(false, NICLA_VIA_ESLOV);
```

with

```
BHY2Host.begin(false, NICLA_VIA_BLE);
```

in the host sketch so that the boards will bind wirelessly.



Bluetooth® Low Energy connection

Using the Nicla Sense ME as an MKR Shield

Another way to communicate the Nicla Sense ME with a Portenta C33/H7 is by using it as a shield.

To convert the Nicla Sense ME into a Shield, you will have to **solder** 2 rows of headers: one side has 9 pins and the other 8 pins; the long side of the headers needs to be on the **battery connectors side.**

The host (Portenta C33/H7) will communicate through the BHY2Host library with the Nicla Sense ME (both

To the Nicla Sense ME to communicate with the Arduino Cloud, set the communication method as `NICLA_AS_SHIELD` in the host sketch as follows:

```
BHY2Host.begin(false, NICLA_AS_SHIELD);
```

Nicla Sense ME as a shield

This setup works with the ESLOV cable too. Keep in mind that female headers or raw cables can be used as well, but make sure the connection of the pin matches the MKR pinout (3V3, GND, SCL and SDA).

For a more detailed process on how to connect the Nicla Sense ME to the Arduino Cloud, follow this **guide**

Support

If you encounter any issues or have questions while working with the Nicla Sense ME, we provide various support resources to help you find answers and solutions.

Help Center

Explore our **Help Center**, which offers a comprehensive collection of articles and guides for the Nicla Sense ME. The Arduino Help Center is designed to provide in-depth

Nicla Family help center page

Forum

Join our community forum to connect with other Nicla Sense ME users, share your experiences, and ask questions. The forum is an excellent place to learn from others, discuss issues, and discover new ideas and projects related to the Nicla Sense ME.

Nicla Sense ME category in the Arduino Forum

Contact Us

Please get in touch with our support team if you need personalized assistance or have questions not covered by the help and support resources described before. We're happy to help you with any issues or inquiries about the Nicla Sense ME.

Contact us page

Was this article helpful?